

Experiment 4: Fibonacci Series (Recursive vs Iterative)

Aim

To implement the **Fibonacci Series** using both **Iterative** and **Recursive** methods in Python.

Algorithm

Iterative Method

Step 1: Start.

Step 2: Read the number of terms n.

Step 3: Initialize a = 0 and b = 1.

Step 4: Print a.

Step 5: Update the values:

- $c = a + b$
- $a = b$
- $b = c$

Step 6: Repeat until n terms are printed.

Step 7: Stop.

Recursive Method

Step 1: Start.

Step 2: Read the number of terms n.

Step 3: If n is 0, return 0.

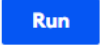
Step 4: If n is 1, return 1.

Step 5: Otherwise return $\text{fib}(n-1) + \text{fib}(n-2)$.

Step 6: Print all Fibonacci terms.

Step 7: Stop.

Source Code (Python)

```
main.py    Share  Run

1 # Fibonacci Series using Iterative and Recursive Methods
2
3 def fibonacci(n):
4     if n <= 1:
5         return n
6     return fibonacci(n - 1) + fibonacci(n - 2)
7
8 n = int(input("Enter number of terms: "))
9
10 # Iterative Method
11 print("Iterative:")
12 a, b = 0, 1
13 for i in range(n):
14     print(a, end=" ")
15     a, b = b, a + b
16
17 print("\nRecursive:")
18 for i in range(n):
19     print(fibonacci(i), end=" ")
```

Output

```
Output

Enter number of terms: 6
Iterative:
0 1 1 2 3 5
Recursive:
0 1 1 2 3 5
=== Code Execution Successful ===
```

Time Complexity

Method Time Complexity Space Complexity

Iterative $O(n)$ $O(1)$

Recursive $O(2^n)$ $O(n)$

Result

The **Fibonacci Series** was generated successfully using both **Iterative** and **Recursive** methods. Both methods produced the same sequence, confirming the correctness of the implementation.

